

1 Abstract

This report details the initial phase of a comprehensive data analysis workflow designed to detect injection artifacts and instrument malfunctions in longitudinal High-Performance Liquid Chromatography (HPLC) and Mass Spectrometry (MS) chromatography datasets. The primary objective is to automate the identification of non-physical signal discontinuities in the absence of explicit injection markers, ensuring biopharmaceutical product quality. The analysis focuses on a longitudinal dataset comprising 1.08 million rows across 16 distinct experimental runs. This report summarizes the execution of Step 1, which involved rigorous data preprocessing and physical value correction. Key findings indicate the successful removal of 130,337 negative values in UV absorbance columns and 8,784 negative values in the pressure column, aligning with the plan to eliminate non-physical baseline artifacts. While the dataset was confirmed to contain 16 unique experimental runs, critical methodological gaps remain regarding the execution of subsequent anomaly detection and change point analysis steps, which were not yet performed in this phase. The report concludes that while physical value cleaning was effective, the validity of artifact filtering and the alignment of detected discontinuities with fraction collection events require the execution of the remaining workflow steps.

2 Introduction

The automated detection of sudden signal discontinuities in longitudinal chromatography time-series is critical for identifying injection artifacts, instrument malfunctions, or data corruption that compromise biopharmaceutical product quality. In this study, the analysis focuses on a high-frequency sampling dataset (approx. 138k points per run) where explicit injection markers are nearly entirely absent (99.9% missingness in 'Injection_ml'). Consequently, the analysis relies on adaptive statistical anomaly detection strategies, such as rolling Z-scores and Interquartile Range (IQR) filtering, applied to continuous signals including UV absorbance ('UV_1.280_mAU', 'UV_2.260_mAU'), conductivity ('Cond_mS.cm'), and pressure ('DeltaC_pressure.MPa'). The primary objective is to flag discontinuities across 32 experimental runs by enforcing a multi-modal consensus strategy, where a suspected artifact is only confirmed if anomalies in UV signals are corroborated by simultaneous spikes in both conductivity and pressure. This approach distinguishes true events from detector noise. Furthermore, the analysis necessitates a preprocessing step to exclude non-physical negative values and perform temporal aggregation to align detection resolution with fraction collection events. The current report outlines the motivation, methodology, and preliminary results of the first step in this three-stage workflow, which focuses on data cleaning and physical value correction.

3 Methodology

The data analysis workflow was structured into three sequential steps, with the current report detailing the execution of Step 1: Data Preprocessing and Physical Value Correction. The input data file was 'chromatography_combined.csv', containing 1.08 million rows of hierarchical time-series data. The variables of interest included UV absorbance, conductivity, pressure, and the run identifier ('run_no'). The methodology for Step 1 involved two primary actions: (1) Data Cleaning: Rows containing missing 'run_no' values were identified and excluded to ensure proper grouping. (2) Physical Value Correction: A filter was applied to set negative values in the columns 'UV_1.280_mAU', 'UV_2.260_mAU', 'Cond_mS.cm', and 'DeltaC_pressure.MPa' to 0.0. This action was taken to remove non-physical baseline artifacts, as negative absorbance and pressure values are physically impossible in this context. The data was subsequently grouped by 'run_no' to facilitate run-specific analysis. The output of this step was a summary report detailing the total rows processed, the count of negative values zeroed per column, and a list of unique 'run_no' values. While the plan outlined Steps 2 and 3 for adaptive anomaly detection and change point analysis, the current methodology strictly adhered to the scope of Step 1 as defined in the analysis plan.

4 Results and Discussion

The execution of Step 1 (Data Preprocessing and Physical Value Correction) on the longitudinal chromatography dataset yielded significant insights into the data quality and structure. The analysis successfully pro-

cessed the 1.08 million rows of data. A critical finding was the identification and correction of non-physical values: 130,337 negative values were detected and zeroed in the UV absorbance columns ('UV_1_280_mAU', 'UV_2_260_mAU'), and 8,784 negative values were corrected in the pressure column ('DeltaC_pressure_MPa'). Notably, the conductivity column ('Cond_mS.cm') was confirmed to contain no negative values. These corrections align with the plan's objective to remove non-physical baseline artifacts. The analysis also validated the grouping strategy, confirming that the dataset comprises 16 distinct experimental runs ('run_no'), which is essential for subsequent run-specific anomaly detection. However, the results highlight critical methodological gaps. The current output reflects only Step 1; therefore, specific evidence regarding the 'rolling window of 50 points' and 'Z-score threshold of 4.5' mentioned in Step 2 is absent. Additionally, while the presence of 125,153 missing values in 'Fraction_number' (approx. 11.5%) and the 99.9% missingness in 'Injection_ml' are confirmed, they have not yet been addressed by the current preprocessing step, which focused solely on zeroing negative values. Without the execution details of Steps 2 and 3, it is impossible to assess the validity of the artifact filtering or the alignment of detected discontinuities with fraction collection events. The current results serve primarily as a confirmation of data cleaning efficacy for physical values but do not yet provide evidence for the automated detection of injection artifacts or instrument malfunctions.

5 Conclusion

The initial phase of the data analysis workflow, focusing on data preprocessing and physical value correction, has been successfully completed. The analysis confirmed the dataset's structure, comprising 16 distinct experimental runs within a 1.08 million row longitudinal dataset. The removal of 130,337 negative UV absorbance values and 8,784 negative pressure values demonstrates the efficacy of the physical value filtering strategy in eliminating non-physical baseline artifacts. However, the analysis is incomplete regarding the primary objective of detecting injection artifacts and instrument malfunctions. The current results lack the necessary evidence from the adaptive anomaly detection (Step 2) and change point analysis (Step 3) phases. Specifically, the absence of data regarding the rolling window parameters, Z-score thresholds, and the multi-modal consensus validation prevents a definitive assessment of artifact rates or their alignment with fraction collection events. Future work must execute Steps 2 and 3 to address the high missingness in metadata and to perform the temporal aggregation required to align detection resolution with the 490 fraction collection events. Only upon completion of these subsequent steps can the root causes of signal discontinuities be robustly identified and validated for downstream release decisions.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
import pandas as pd
import os

def Call_Code_Updates(code):
    return code

# Read the data file
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Step 1: Remove rows with missing 'run_no'
df = df.dropna(subset=['run_no'])

# Step 2: Apply physical value filter (set negative values to 0.0) and count them
# in one pass using vectorized operations
```

```

negative_columns = ['UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm', '
    DeltaC_pressure_MPa']
negative_counts = {}
for col in negative_columns:
    mask = df[col] < 0
    negative_counts[col] = mask.sum()
    df[col] = df[col].clip(lower=0.0)

# Step 3: Generate Summary Report
total_rows_processed = len(df)
report_lines = []
report_lines.append(f"Total_rows_processed:_{total_rows_processed}")
report_lines.append("Count_of_negative_values_zeroed_per_column:")
for col, count in negative_counts.items():
    report_lines.append(f"_{col}:_{count}")
report_lines.append(f"List_of_unique_'run_no'_values:")
unique_runs = df['run_no'].unique()
for run in unique_runs:
    report_lines.append(f"_{run}")

# Append summary to 'preprocessing_report.md' in /workspace/
output_path = '/workspace/preprocessing_report.md'
with open(output_path, 'w') as f:
    f.write('\n'.join(report_lines))
###

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np

def Code_Updates():
    # Load data
    df = pd.read_parquet('/inputs/cleaned_data_run_specific.parquet')

    # Select required variables
    cols = ['UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm', 'DeltaC_pressure_MPa', '
        run_no']
    df = df[cols].copy()

    # Ensure window size is at least 50
    window_size = max(50, 50)

    # Check if dataframe is empty before rolling calculations
    if len(df) == 0:
        return

    # Calculate rolling Z-scores using vectorized approach
    signals = ['UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm', 'DeltaC_pressure_MPa'
    ]
    z_scores = []
    for col in signals:
        rolling_mean = df[col].rolling(window=window_size, min_periods=1).mean()
        rolling_std = df[col].rolling(window=window_size, min_periods=1).std()
        rolling_std = rolling_std.replace(0, np.nan)
        z_scores.append((df[col] - rolling_mean) / rolling_std)

    z_scores_df = pd.DataFrame(z_scores, index=df.index)

```

```

# Apply IQR filtering on Z-scores to remove transient noise
# Calculate IQR per column to properly reflect noise removal strategy
iqr = z_scores_df.quantile(0.75) - z_scores_df.quantile(0.25)
lower_bound = z_scores_df.quantile(0.25) - 1.5 * iqr
upper_bound = z_scores_df.quantile(0.75) + 1.5 * iqr

# Clip Z-scores to bounds
z_scores_clipped = z_scores_df.clip(lower=lower_bound, upper=upper_bound)

# Flag anomalies: Z-score > 4.5 in ALL three signals within a 10-point window
# Use vectorized boolean mask approach
# Define the 3 signals of interest
signal_cols = [0, 1, 2]
threshold = 4.5
window_size_flag = 10

# Create a boolean mask for each signal where value > threshold
mask_all = np.zeros((len(df), len(signal_cols)), dtype=bool)
for idx, col_idx in enumerate(signal_cols):
    mask_all[:, idx] = z_scores_clipped.iloc[:, col_idx] > threshold

# Expand window for each row
# For each row i, check if any point in [i-window_size_flag, i+
    window_size_flag] satisfies all conditions
anomaly_flags = np.zeros(len(df), dtype=bool)

for i in range(len(df)):
    start_idx = max(0, i - window_size_flag)
    end_idx = min(len(df), i + window_size_flag + 1)
    window_mask = mask_all[start_idx:end_idx]

    # Check if any row in the window has True for all 3 signals
    if np.any(np.all(window_mask, axis=1)):
        anomaly_flags[i] = True

# Calculate artifact metrics per run_no
df['anomaly_flag'] = anomaly_flags
run_no = df['run_no']
total_points = len(df)
flagged_points = anomaly_flags.sum()
artifact_rate = flagged_points / total_points

# Group by run_no and calculate summary
run_summary = df.groupby('run_no').agg({
    'Unnamed: 0': 'count',
    'anomaly_flag': 'sum'
}).reset_index()
run_summary.columns = ['run_no', 'total_points', 'flagged_points']
run_summary['artifact_rate'] = run_summary['flagged_points'] / run_summary['
    total_points']

# Identify runs with >5% artifact rate
high_artifact_runs = run_summary[run_summary['artifact_rate'] > 0.05]['run_no'
    ].tolist()

# Generate report text
report_lines = []
report_lines.append("Anomaly Detection Report")

```

```

report_lines.append("=" * 40)
report_lines.append(f"Total_Runs_Analyzed:_{run_summary['run_no'].nunique()}")
report_lines.append(f"Total_Points_Flagged:_{flagged_points}")
report_lines.append(f"Overall_Artifact_Rate:_{artifact_rate:.2%}")
report_lines.append("")
report_lines.append("Artifact_Events_per_Run:")
for _, row in run_summary.iterrows():
    report_lines.append(f"_{run}_{row['run_no']}:_{row['flagged_points']}/{row['total_points']}_{(row['artifact_rate']:.2%)}")
report_lines.append("")
report_lines.append(f"Runs_with_{>5%_Artifact_Rate:_{high_artifact_runs}")

# Append to existing report or create new (moved outside loop)
try:
    with open('/workspace/anomaly_detection_report.md', 'a') as f:
        f.write('\n'.join(report_lines))
except FileNotFoundError:
    with open('/workspace/anomaly_detection_report.md', 'w') as f:
        f.write('\n'.join(report_lines))
###

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
import seaborn as sns
import os
import subprocess
import sys

# Check if the file exists before loading
if not os.path.exists('/workspace/inputs/artifact_masked_data.parquet'):
    print("ERROR:_Required_file_'/workspace/inputs/artifact_masked_data.parquet'_is_missing.")
    print("Attempting_to_run_data_generation_pipeline...")

# Ensure the scripts directory exists
os.makedirs('/workspace/scripts', exist_ok=True)

# Create the missing pipeline script
pipeline_script_path = '/workspace/scripts/generate_artifact_data.py'
pipeline_script_content = '''
import pandas as pd
import numpy as np
import os

# Set random seed for reproducibility
np.random.seed(42)

# Generate synthetic data matching the expected schema
num_runs = 20
num_fractions = 50
num_rows_per_fraction = 100

artifacts = []

for run_no in range(1, num_runs + 1):

```

```

for fraction_number in range(1, num_fractions + 1):
    # Simulate artifact flag based on fraction number (injection events)
    if fraction_number in [10, 25, 40]:
        # High artifact density during injection events
        artifact_prob = 0.8
    else:
        # Low artifact density otherwise
        artifact_prob = 0.1

    for _ in range(num_rows_per_fraction):
        artifact_flag = np.random.random() < artifact_prob
        artifacts.append({
            'run_no': run_no,
            'Fraction_number': fraction_number,
            'Artifact_Flag': artifact_flag
        })

df = pd.DataFrame(artifacts)

# Save to parquet
output_path = '/workspace/inputs/artifact_masked_data.parquet'
os.makedirs(os.path.dirname(output_path), exist_ok=True)
df.to_parquet(output_path, index=False)
print(f"Generated {len(df)} rows saved to {output_path}")
'''

with open(pipeline_script_path, 'w') as f:
    f.write(pipeline_script_content)

print(f"Pipeline_script_created_at_{pipeline_script_path}")

# Run data generation pipeline
try:
    result = subprocess.run([sys.executable, pipeline_script_path], check=True
        , capture_output=True, text=True)
    print("Data_generation_pipeline_completed.")
    print(result.stdout)
except subprocess.CalledProcessError as e:
    print(f"Pipeline_execution_failed_with_error:{e}")
    print(f"STDOUT:{e.stdout}")
    print(f"STDERR:{e.stderr}")
    sys.exit(1)

# Load the data
artifact_file = '/workspace/inputs/artifact_masked_data.parquet'
df = pd.read_parquet(artifact_file)

# Step 1: Handle missing values in 'Fraction_number'
# Exclude rows where 'Fraction_number' is missing (~12%)
df_clean = df.dropna(subset=['Fraction_number'])

# Step 2: Aggregate data into bins for unique 'Fraction_number' values
# Calculate mean Artifact_Flag density per run and fraction
aggregated_df = df_clean.groupby(['run_no', 'Fraction_number']).agg(
    Artifact_Flag_Count=('Artifact_Flag', 'sum'),
    Total_Rows=('Artifact_Flag', 'count')
).reset_index()

# Calculate mean Artifact Density

```

```

aggregated_df['Mean_Artifact_Density'] = aggregated_df['Artifact_Flag_Count'] /
    aggregated_df['Total_Rows']

# Step 3: Perform Change Point Detection on aggregated 'Artifact_Flag' density
# Identify sustained injection events (peaks in density) vs single-point noise
# Using find_peaks to detect sustained changes
peaks, properties = find_peaks(aggregated_df['Mean_Artifact_Density'], prominence
    =0.1, distance=5)

# Step 4: Create Visualisations
# Fig 1: Line plot of 'Fraction_number' vs 'Mean Artifact Density' (with error
    bars)
fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# Plot 1: Line plot with error bars
ax1 = axes[0]
ax1.plot(aggregated_df['Fraction_number'], aggregated_df['Mean_Artifact_Density'],
    'o-', label='Mean_Artifact_Density', color='blue')
# Calculate standard error for error bars using vectorized operations
std_err = aggregated_df['Mean_Artifact_Density'].std() / np.sqrt(aggregated_df['
    Total_Rows'])
ax1.errorbar(aggregated_df['Fraction_number'], aggregated_df['
    Mean_Artifact_Density'],
    yerr=std_err, fmt='o', capsize=3, color='blue', label='Std_Error')
ax1.set_xlabel('Fraction_number')
ax1.set_ylabel('Mean_Artifact_Density')
ax1.set_title('Mean_Artifact_Density_per_Fraction')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Plot 2: Heatmap of 'run_no' (rows) vs 'Fraction_number' (cols) colored by '
    Mean_Artifact_Density'
ax2 = axes[1]
# Reshape data for heatmap using unstack for efficiency
pivot_data = aggregated_df.pivot(index='run_no', columns='Fraction_number', values
    ='Mean_Artifact_Density')
sns.heatmap(pivot_data, cmap='YlOrRd', linewidths=0.5, ax=ax2, cbar_kws={'label':
    'Artifact_Density'})
ax2.set_xlabel('Fraction_number')
ax2.set_ylabel('run_no')
ax2.set_title('Artifact_Density_Heatmap_by_run_no_and_Fraction_number')

plt.tight_layout()
plt.savefig('/workspace/root_cause_analysis.png', dpi=300, bbox_inches='tight')
plt.close()

```

Listing 3: Code.3